批量生产问题解决方案

钟珊珊 22214624

目录

(一) 问题描述 1
(二) 实验探究 1
(三) 代码说明 6

(一) 问题描述

批量生产（batch manufacturing）问题描述了不断有新的货物进入仓库，处理和存储货物都需要开销，需要找到一个开销最小的货物处理策略。问题的具体定义如下所示：

- A Manufacturer at each time period receives an order for her product with probability p and receives no order with probability $1-p$
- At any period, she has a choice of processing all the unfilled orders in a batch, or process no order at all. The maximum number of orders that can remain unfilled is n
- The cost per unfilled order at each time period is $c > 0$, the setup cost to process the unfilled orders is $K > 0$
- The manufacturer wants to find a processing policy that minimizes the total expected cost with discount factor $\alpha < 1$

(二) 实验探究

2.1 Run the code with different parameter inputs to show the performance of value iteration and policy iteration.

由题意，定义如下基本变量：

- State $i \in \{0, 1, \cdots, n\}$: number of unfilled orders
- Action $u \in \{0, 1\}$: process (1) or not (0)

$$u \in \{0, 1\}, \text{if } i < n; \quad u = 1, \text{if } i = n$$

- State Transition $p_{ij}(u)$:

$$p_{i1}(1) = p_{i(i+1)}(0) = p, \quad p_{i0}(1) = p_{ii}(0) = 1 - p, \quad i < n$$

$$p_{n1}(1) = p, \quad p_{n0}(1) = 1 - p$$

- Per-stage cost

$$g(i, 1) = K, \quad g(i, 0) = ci$$

将最优处理时间定义为 t，下面分点讨论 discounted 和 average cost 问题。

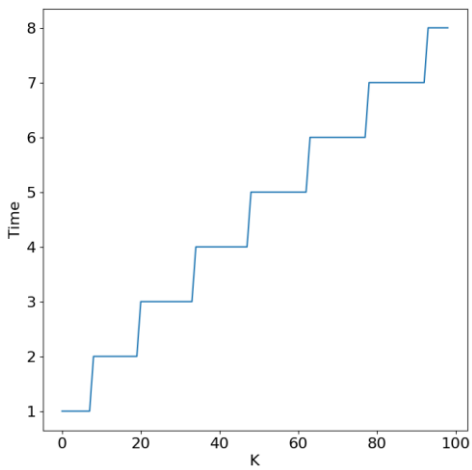
2.1.1 Discounted problem

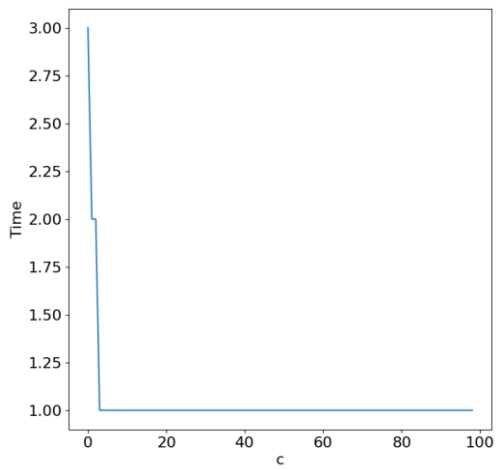
（1） Value iteration

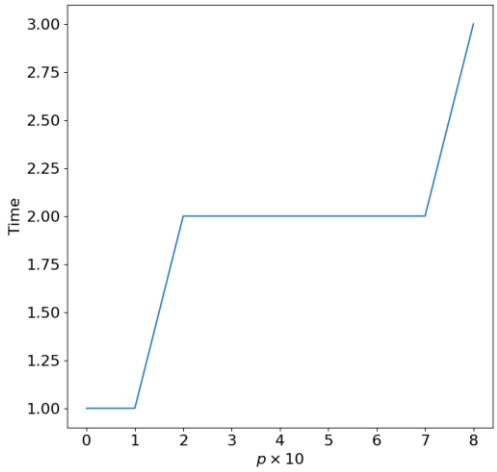
建立如下 bellman 方程：

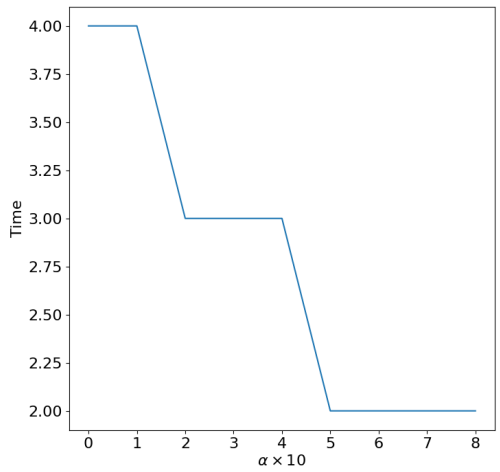
$$\begin{aligned} J_{k+1}(i) = \min \big[& K + \alpha(1-p)J_k(0) + \alpha p J_k(1), \\ & ci + \alpha(1-p)J_k(i) + \alpha p J_k(i+1) \big], \quad i = 0, 1, \cdots, n-1, \\ J_{k+1}(n) = & K + \alpha(1-p)J_k(0) + \alpha p J_k(1) \end{aligned}$$

以 K = 10，c = 3，n = 10，p = 0.2，α = 0.8 为默认参数，不同参数下阈值的实验结果如下：



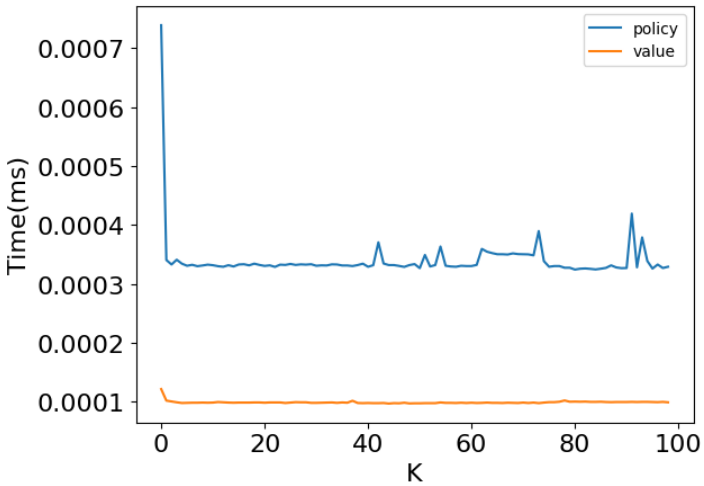






（2） **Policy iteration**

该方法得到结果与 value iteration 一致。如下图所示，以 K 为例，我们可以发现 policy iteration 需要消耗更多时间。



2.1.2 Average cost problem

该问题可以建模如下：

$$\begin{aligned}\lambda^* + h^*(i) &= \min\big[K + (1-p)h^*(0) + ph^*(1),\\ &\qquad ci + (1-p)h^*(i) + ph^*(i+1)\big], \quad i = 0, 1, \cdots, n-1,\\ \lambda^* + h^*(n) &= K + (1-p)h^*(0) + ph^*(1)\end{aligned}$$

（1） **Value iteration**

$$\lambda^* + h^*(i) = \min_{u \in U(i)} \Big[g(i,u) + \sum_{j=1}^{n} p_{ij}(u)h^*(j)\Big], \forall i$$

- Unknown parameters $\lambda^*, h^*(i)$
- Relative value iteration
  Given $h_k(i)$, and for some fixed state s, calculate

$$\lambda_k = \min_{u \in U(s)} \left[g(s,u) + \sum_{j=1}^{n} p_{sj}(u) h_k(j) \right]$$

and

$$h_{k+1}(i) = \min_{u \in U(i)} \left[g(i,u) + \sum_{j=1}^{n} p_{ij}(u) h_k(j) \right] - \lambda_k$$

（2） Policy iteration

- At iteration k, we have a stationary μ^k
- Policy iteration: Solving linear equations

$$\lambda^k + h^k(i) = g(i, \mu^k(i)) + \sum_{j=1}^{n} p_{ij}(\mu^k(i)) h^k(j), \forall i$$

$$h^k(n) = 0$$

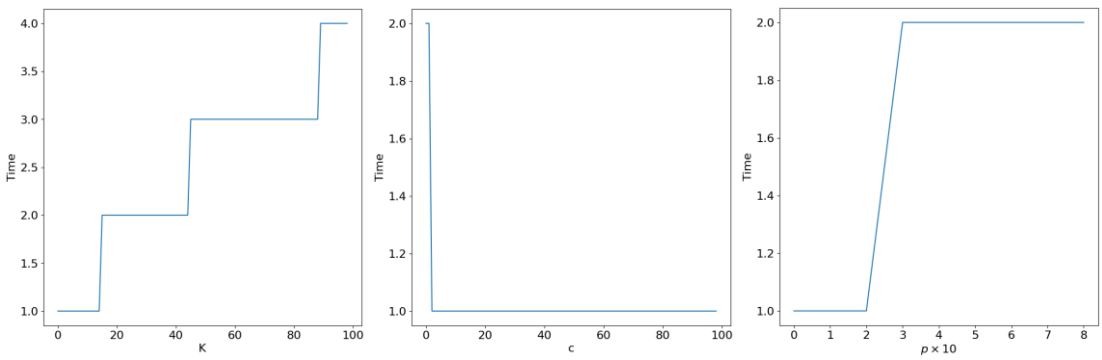
- Policy improvement:

$$\mu^{k+1}(i) = \arg\min_{u \in U(i)} \left[g(i,u) + \sum_{j=1}^{n} p_{ij}(u) h^k(j) \right], \forall i$$

- Terminate when $\lambda^{k+1} = \lambda^k, h^{k+1}(i) = h^k(i), \forall i$

（3） 实验结果

以 K = 10，c = 3，n = 10，p = 0.2 为默认参数，不同参数变化下阈 value iteration 和 policy iteration 的阈值结果一致，均如下图所示，但是同样地，policy iteration 的计算速度相对较慢。



2.2 For discounted problem, consider exercise 7.8.

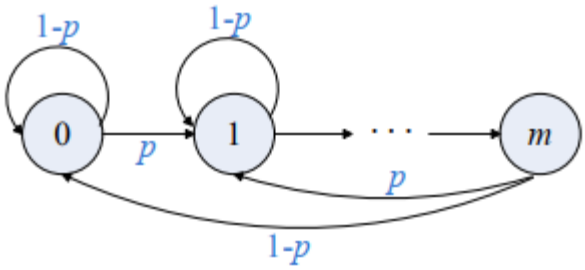
Consider the policy iteration algorithm for the problem of Example 7.3.2.

(a) Show that if we start the algorithm with a threshold policy, every subsequently generated policy will be a threshold policy. *Note*: This requires a careful argument.

(b) Carry out the algorithm for the case $c = 1$, $K = 5$, $n = 10$, $p = 0.5$, $\alpha = 0.9$, and an initial policy that always processes the unfilled orders.

(a) 证明参考 Dynamic Programming and Optimal Control- Volume 1- SECOND EDITION 的 solution chapter7-7.8；(b) 当 c=1，K=5，n=10，p=0.5，α=0.9 时，带入代码可以得到 J 的值为[14.625, 17.875, 19.625, 19.625, 19.625, 19.625, 19.625, 19.625, 19.625, 19.625, 19.625]，故阈值为 2。

2.3 For average cost problem, write code to find the threshold by stationary analysis.



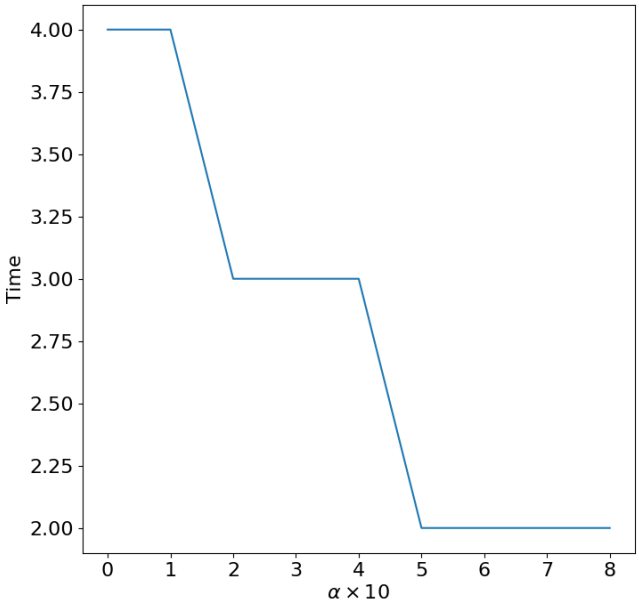
根据如上图所示的转移概率图，可以得到如下转移概率矩阵 P。接着，我们可以通过 $\pi_j = \sum_{k=1}^{M} \pi_k P_{kj}$. 计算 P 的平稳分布，并通过下式计算 f(m)：

$$\sum_{i=0}^{m-1} ci\pi_i + K\pi_m := f(m)$$

以 K = 10，c = 3，n = 10，p = 0.2 为参数，可以计算得到 f(m)分布，可知 m=1 时 f(m)值最小。

2.4 Compare the optimal policies of the discounted problem with different values of α and the average cost problem.

以 K = 10，c = 3，n = 10，p = 0.2 为参数，计算结果如下：



(三) 代码说明

3.1 Discounted problem + value iteration (discounted_value.py)

```
import numpy as np

K = 10
c = 3
iteration = 10
n = 3
p = 0.2
alpha = 0.8

J = np.random.rand(n + 1)
for it in range(iteration):
    for i in range(n):
        J[i] = min(
            K + alpha * (1 - p) * J[0] + alpha * p * J[1],
            c * i + alpha * (1 - p) * J[i] + alpha * p * J[i + 1],
```

```
        )
    J[-1] = K + alpha * (1 - p) * J[0] + alpha * p * J[1]
```

3.2 Discounted problem + policy iteration (discounted _policy.py)

```
import numpy as np

K = 10
c = 3
iteration = 5
n = 3
p = 0.2
alpha = 0.8

def g(s, action):
    if action == 1:
        return K
    if action == 0:
        return c * s

mu = np.ones(n + 1)
M = np.zeros((n + 1, n + 1))

for itera in range(iteration):
    M = np.zeros((n + 1, n + 1))
    M[n][n], M[n][0], M[n][1] = 1, -alpha * (1 - p), -alpha * p
    for i in range(n):
        if mu[i] == 0:
            M[i][i] = 1 - alpha * (1 - p)
```

```
            M[i][i + 1] = -alpha * p
        else:
            if i == 0:
                M[0][0] = 1 - alpha * (1 - p)
                M[i][1] = -alpha * p
            else:
                M[i][i], M[i][0], M[i][1] = 1, -alpha * (1 - p), -alpha * p

    G = np.array([g(i, mu[i]) for i in range(n)] + [K])
    J = np.dot(np.linalg.inv(M), G)

    for i in range(n):
        cost1 = K + alpha * (1 - p) * J[0] + alpha * p * J[1]
        cost0 = c * i + alpha * (1 - p) * J[i] + alpha * p * J[i + 1]
        if cost1 > cost0:
            mu[i] = 0
        else:
            mu[i] = 1
```

3.3 Average cost problem + value iteration (avg_cost_value.py)

```
import numpy as np

K = 10
c = 3
iteration = 10
n = 3
p = 0.2
alpha = 1
```

```
h0 = np.zeros(n + 1)
lambdax = 0
for it in range(iteration):
    h = h0.copy()
    for i in range(n):
        h0[i] = (
            min(
                K + alpha * (1 - p) * h[0] + alpha * p * h[1],
                c * i + alpha * (1 - p) * h[i] + alpha * p * h[i + 1],
            )
            - lambdax
        )
    h0[-1] = K + alpha * (1 - p) * h[0] + alpha * p * h[1] - lambdax

    tmp = [
        c * i + alpha * (1 - p) * h0[i] + alpha * p * h0[i + 1]
        for i in range(n)
    ]
    tmp.append(K + alpha * (1 - p) * h0[0] + alpha * p * h0[1])
    lambdax = min(tmp)
```

3.4 Average cost problem + policy iteration (avg_cost_policy.py)

```
import numpy as np

K = 10
c = 3
iteration = 10
```

```
n = 3
p = 0.2
alpha = 1

def g(s, action):
    if action == 1:
        return K
    if action == 0:
        return c * s

lambdax = 0
mu = np.ones(n + 1)
M = np.zeros((n + 1, n + 1))
h = np.zeros(n + 1)

for itera in range(iteration):
    M = np.zeros((n + 2, n + 2))
    M[n][n], M[n][0], M[n][1], M[n + 1, n], M[n - 1, n + 1] = (
        1,
        -alpha * (1 - p),
        -alpha * p,
        1,
        1,
    )
    for i in range(n):
        M[i][n + 1] = 1
        if mu[i] == 0:
            M[i][i] = 1 - alpha * (1 - p)
```

```
            M[i][i + 1] = -alpha * p
        else:
            if i == 0:
                M[0][0] = 1 - alpha * (1 - p)
                M[i][1] = -alpha * p
            else:
                M[i][i], M[i][0], M[i][1] = 1, -alpha * (1 - p), -alpha * p

    G = np.array([g(i, mu[i]) for i in range(n)] + [K] + [0])
    J = np.dot(np.linalg.inv(M), G)
    h, lambdax = J[:-1], J[-1]

    for i in range(n):
        cost1 = K + alpha * (1 - p) * J[0] + alpha * p * J[1]
        cost0 = c * i + alpha * (1 - p) * J[i] + alpha * p * J[i + 1]
        if cost1 > cost0:
            mu[i] = 0
        else:
            mu[i] = 1
    mu[n] = 1
```

3.5 其他分析代码及示例 (analysis.ipynb)